PRACTICAL FILE

Open Source Tools and Frameworks

Python Programming — Practical Programs with Code and Output

Course	IMCA
Subject Code	05201333
Subject	Open Source Tools and Frameworks
Faculty	Prof. Adarsh Ashok
Institution	Parul University, Waghodia, Vadodara
Programming Language	Python 3

LIST OF PRACTICALS

1. Installation and Configuration of Python
2. Branching Programs — Largest of Two Numbers
3. Looping Programs — String Length without len()
4. Python Functions — Basic Calculator
5. File Handling — Phone List Application
6. Lists — Merge, Sort and Remove Most Repeated Element
7. Tuples — All Possible Operations
8. Dictionary — Create, Add, Delete, Update, Print

PRACTICAL 1

Installation and Configuration of Python

AIM

Understand installation and configuration of Python. Write a simple Python program which adds two integer values and execute it using IDE.

THEORY / CONCEPT

Python can be installed from the official website **python.org** by downloading the installer for Windows/Linux/macOS. During installation on Windows, the option **“Add Python to PATH”** must be checked so that Python can be run from the command line. Installation can be verified by running `python --version` in the command prompt / terminal.

Python programs can be written and executed using an IDE such as **IDLE** (which comes bundled with Python), **PyCharm**, **VS Code**, or **Thonny**. A program is saved with a `.py` extension and executed either by pressing **Run** in the IDE or by typing `python filename.py` in the terminal.

PROGRAM / SOURCE CODE

```
# Practical 1: Installation and Configuration of Python
# Simple Python program to add two integer values

num1 = 10
num2 = 20

sum_result = num1 + num2

print("Number 1:", num1)
print("Number 2:", num2)
print("Sum of the two numbers is:", sum_result)
```

OUTPUT

```
Number 1: 10
Number 2: 20
Sum of the two numbers is: 30
```

PRACTICAL 2

Branching Programs

AIM

Write a Python program which takes two numbers as input. Use the if-then-else construct to determine the largest of them (integers, floats, etc.), and check for different outputs.

THEORY / CONCEPT

The `if-elif-else` construct in Python allows a program to make decisions and execute different blocks of code based on a condition. Python does not require explicit type declaration, so the same comparison logic (>, <, ==) works seamlessly on **int**, **float**, and other numeric data types, since Python performs implicit type conversion during comparison between numeric types.

PROGRAM / SOURCE CODE

```
# Practical 2: Branching Programs
# Find the largest of two numbers using if-then-else
# Works for int, float, and other numeric types

def find_largest(a, b):
    if a > b:
        return a
    elif b > a:
        return b
    else:
        return a  # both are equal

# ---- Test Case 1: Integer values ----
num1 = 45
num2 = 89
print("Test Case 1: Integers")
print("Number 1 =", num1, " Number 2 =", num2)
print("Largest number is:", find_largest(num1, num2))
print()

# ---- Test Case 2: Float values ----
num3 = 23.75
num4 = 23.5
print("Test Case 2: Floats")
print("Number 1 =", num3, " Number 2 =", num4)
print("Largest number is:", find_largest(num3, num4))
print()

# ---- Test Case 3: Equal values ----
num5 = 15
num6 = 15
print("Test Case 3: Equal Numbers")
print("Number 1 =", num5, " Number 2 =", num6)
print("Largest number is:", find_largest(num5, num6))
print()

# ---- Test Case 4: Mixed int and float ----
num7 = 10
num8 = 10.5
print("Test Case 4: Mixed int and float")
print("Number 1 =", num7, " Number 2 =", num8)
print("Largest number is:", find_largest(num7, num8))

# ---- Interactive input version ----
# Uncomment below for taking values from the user at runtime:
# a = float(input("Enter first number: "))
# b = float(input("Enter second number: "))
# print("Largest number is:", find_largest(a, b))
```

OUTPUT

```
Test Case 1: Integers
Number 1 = 45  Number 2 = 89
Largest number is: 89

Test Case 2: Floats
Number 1 = 23.75  Number 2 = 23.5
Largest number is: 23.75

Test Case 3: Equal Numbers
Number 1 = 15  Number 2 = 15
Largest number is: 15
```

```
Test Case 4: Mixed int and float
Number 1 = 10  Number 2 = 10.5
Largest number is: 10.5
```

PRACTICAL 3

Looping Programs

AIM

Write a program which takes a string as input. Calculate the length of a given string (without using the built-in len() function).

THEORY / CONCEPT

Looping constructs such as `for` and `while` allow a block of code to be executed repeatedly. In Python, a string is an iterable sequence of characters, so a `for` loop can traverse each character one by one. By initializing a counter variable to zero and incrementing it once for every character encountered in the loop, the length of the string can be calculated manually without relying on the built-in `len()` function.

PROGRAM / SOURCE CODE

```
# Practical 3: Looping Programs
# Calculate the length of a given string WITHOUT using the built-in len() function

def string_length(s):
    count = 0
    for ch in s:
        count = count + 1
    return count

# ---- Test Case 1 ----
str1 = "Parul University"
print("String 1:", str1)
print("Length of string 1:", string_length(str1))
print()

# ---- Test Case 2 ----
str2 = "IoT"
print("String 2:", str2)
print("Length of string 2:", string_length(str2))
print()

# ---- Test Case 3: Empty string ----
str3 = ""
print("String 3: (empty string)")
print("Length of string 3:", string_length(str3))
print()

# ---- Test Case 4: String with spaces ----
str4 = "Open Source Tools and Frameworks"
print("String 4:", str4)
print("Length of string 4:", string_length(str4))

# ---- Interactive input version ----
# user_str = input("Enter a string: ")
# print("Length of the string is:", string_length(user_str))
```

OUTPUT

```
String 1: Parul University
Length of string 1: 16

String 2: IoT
Length of string 2: 3

String 3: (empty string)
Length of string 3: 0

String 4: Open Source Tools and Frameworks
Length of string 4: 32
```

PRACTICAL 4

Python Functions

AIM

Write a program to create a basic calculator using functions. The program must be able to perform addition, subtraction, multiplication and division of any two compatible data types.

THEORY / CONCEPT

Functions in Python are defined using the def keyword and allow code to be organized into reusable, logical blocks. In this program, separate functions are created for each arithmetic operation (`add()`, `subtract()`, `multiply()`, `divide()`), and a controller function `calculator()` decides which operation function to call based on the operator symbol passed by the user. Division by zero is handled explicitly to avoid runtime errors.

PROGRAM / SOURCE CODE

```
# Practical 4: Python Functions
# Basic calculator using functions - supports addition, subtraction,
# multiplication and division on any compatible data types (int, float)

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

def multiply(a, b):
    return a * b

def divide(a, b):
    if b == 0:
        return "Error: Division by zero is not allowed"
    return a / b

def calculator(a, b, operation):
    if operation == '+':
        return add(a, b)
    elif operation == '-':
        return subtract(a, b)
    elif operation == '*':
        return multiply(a, b)
    elif operation == '/':
        return divide(a, b)
    else:
        return "Error: Invalid operation"

# ---- Test Cases ----
print("Test Case 1: Integers")
a, b = 20, 5
print(f"{a} + {b} =", calculator(a, b, '+'))
print(f"{a} - {b} =", calculator(a, b, '-'))
print(f"{a} * {b} =", calculator(a, b, '*'))
print(f"{a} / {b} =", calculator(a, b, '/'))
print()

print("Test Case 2: Floats")
a, b = 12.5, 2.5
print(f"{a} + {b} =", calculator(a, b, '+'))
print(f"{a} - {b} =", calculator(a, b, '-'))
print(f"{a} * {b} =", calculator(a, b, '*'))
print(f"{a} / {b} =", calculator(a, b, '/'))
print()

print("Test Case 3: Mixed int and float")
a, b = 15, 4.0
print(f"{a} + {b} =", calculator(a, b, '+'))
print(f"{a} / {b} =", calculator(a, b, '/'))
print()

print("Test Case 4: Division by zero")
a, b = 10, 0
print(f"{a} / {b} =", calculator(a, b, '/'))

# ---- Interactive input version ----
# x = float(input("Enter first number: "))
# y = float(input("Enter second number: "))
# op = input("Enter operation (+, -, *, /): ")
# print("Result:", calculator(x, y, op))
```

OUTPUT

```
Test Case 1: Integers
20 + 5 = 25
20 - 5 = 15
20 * 5 = 100
20 / 5 = 4.0

Test Case 2: Floats
12.5 + 2.5 = 15.0
12.5 - 2.5 = 10.0
12.5 * 2.5 = 31.25
12.5 / 2.5 = 5.0

Test Case 3: Mixed int and float
15 + 4.0 = 19.0
15 / 4.0 = 3.75

Test Case 4: Division by zero
10 / 0 = Error: Division by zero is not allowed
```

PRACTICAL 5

File Handling

AIM

Write a program to create a phone list. Take person name and phone number as input from the user. Create a class Phone_list which has add_number(), lookup(), remove_contact(). add_number() saves details in a text file; lookup() finds a contact number by name from the saved file.

THEORY / CONCEPT

File handling in Python is performed using the built-in `open()` function, which supports modes such as `'r'` (read), `'a'` (append), and `'w'` (write). Using Object-Oriented Programming, a class `Phone_list` is designed to encapsulate all phonebook-related operations. The `add_number()` method appends a new contact as a line of comma-separated values in a text file. The `lookup()` method reads the file line by line and searches for a matching contact name. The `remove_contact()` method reads all existing records, filters out the one to be deleted, and rewrites the file — effectively deleting that record from persistent storage.

PROGRAM / SOURCE CODE

```
# Practical 5: File Handling
# Phone list application using a class with add_number(), lookup(),
# and remove_contact() methods. Data is stored in a text file.

class Phone_list:
    def __init__(self, filename="contacts.txt"):
        self.filename = filename
        # create the file if it does not already exist
        open(self.filename, "a").close()

    def add_number(self, name, number):
        with open(self.filename, "a") as f:
            f.write(f"{name},{number}\n")
        print(f"Contact added: {name} -> {number}")

    def lookup(self, name):
        with open(self.filename, "r") as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                saved_name, saved_number = line.split(",")
                if saved_name.lower() == name.lower():
                    print(f"Found: {saved_name} -> {saved_number}")
                    return saved_number
        print(f"No contact found with name: {name}")
        return None

    def remove_contact(self, name):
        with open(self.filename, "r") as f:
            lines = f.readlines()

        found = False
        with open(self.filename, "w") as f:
            for line in lines:
                saved_name = line.split(",")[0].strip()
                if saved_name.lower() == name.lower():
                    found = True
                    continue  # skip writing this line, i.e. delete it
                f.write(line)

        if found:
            print(f"Contact removed: {name}")
        else:
            print(f"No contact found with name: {name}")

    def show_all(self):
        with open(self.filename, "r") as f:
            content = f.read()
        print("---- Current Contact List ----")
        print(content if content else "(empty)")


# ---- Demonstration ----
phonebook = Phone_list("contacts.txt")

print("Adding contacts:")
phonebook.add_number("Aadhi", "9998887770")
phonebook.add_number("Rahul", "9123456780")
phonebook.add_number("Priya", "8877665544")
print()

phonebook.show_all()
print()

print("Looking up contacts:")
```

```
phonebook.lookup("Rahul")
phonebook.lookup("Sneha")   # not present
print()

print("Removing a contact:")
phonebook.remove_contact("Rahul")
print()

phonebook.show_all()

# ---- Interactive input version ----
# phonebook = Phone_list("contacts.txt")
# name = input("Enter name: ")
# number = input("Enter phone number: ")
# phonebook.add_number(name, number)
# search_name = input("Enter name to search: ")
# phonebook.lookup(search_name)
# del_name = input("Enter name to remove: ")
# phonebook.remove_contact(del_name)
```

OUTPUT

```
Adding contacts:
Contact added: Aadhi -> 9998887770
Contact added: Rahul -> 9123456780
Contact added: Priya -> 8877665544

---- Current Contact List ----
Aadhi,9998887770
Rahul,9123456780
Priya,8877665544


Looking up contacts:
Found: Rahul -> 9123456780
No contact found with name: Sneha

Removing a contact:
Contact removed: Rahul

---- Current Contact List ----
Aadhi,9998887770
Priya,8877665544
```

PRACTICAL 6

Lists

AIM

Write a program which merges the content of two lists, sorts the merged list, and removes the element which is repeated the maximum number of times.

THEORY / CONCEPT

Lists in Python are mutable, ordered collections. The + operator is used to concatenate (merge) two lists, and the `sort()` method arranges the elements in ascending order. To find the most frequently repeated element, a frequency dictionary is built by counting the occurrences of each element while traversing the merged list. The element with the highest count is then removed from the list using a list comprehension that filters it out.

PROGRAM / SOURCE CODE

```
# Practical 6: Lists
# Merge two lists, sort the merged list, and remove the element
# which is repeated the maximum number of times

def merge_sort_and_remove_most_frequent(list1, list2):
    merged = list1 + list2
    merged.sort()

    # Count frequency of each element without using Counter
    frequency = {}
    for item in merged:
        if item in frequency:
            frequency[item] += 1
        else:
            frequency[item] = 1

    # Find the element with the maximum frequency
    max_freq = 0
    most_repeated = None
    for item, freq in frequency.items():
        if freq > max_freq:
            max_freq = freq
            most_repeated = item

    # Remove all occurrences of the most repeated element
    result = [item for item in merged if item != most_repeated]

    return merged, most_repeated, max_freq, result


# ---- Test Case 1 ----
list1 = [5, 3, 8, 3, 1, 9]
list2 = [2, 3, 7, 4, 3, 6]

merged, most_repeated, freq, result = merge_sort_and_remove_most_frequent(list1, list2)

print("Test Case 1")
print("List 1:", list1)
print("List 2:", list2)
print("Merged and Sorted List:", merged)
print(f"Most repeated element: {most_repeated} (appeared {freq} times)")
print("List after removing most repeated element:", result)
print()

# ---- Test Case 2 ----
list3 = [10, 20, 20, 30]
list4 = [40, 20, 50, 10]

merged2, most_repeated2, freq2, result2 = merge_sort_and_remove_most_frequent(list3, list4)

print("Test Case 2")
print("List 3:", list3)
print("List 4:", list4)
print("Merged and Sorted List:", merged2)
print(f"Most repeated element: {most_repeated2} (appeared {freq2} times)")
print("List after removing most repeated element:", result2)
```

OUTPUT

```
Test Case 1
List 1: [5, 3, 8, 3, 1, 9]
List 2: [2, 3, 7, 4, 3, 6]
Merged and Sorted List: [1, 2, 3, 3, 3, 3, 4, 5, 6, 7, 8, 9]
Most repeated element: 3 (appeared 4 times)
```

```
List after removing most repeated element: [1, 2, 4, 5, 6, 7, 8, 9]

Test Case 2
List 3: [10, 20, 20, 30]
List 4: [40, 20, 50, 10]
Merged and Sorted List: [10, 10, 20, 20, 20, 30, 40, 50]
Most repeated element: 20 (appeared 3 times)
List after removing most repeated element: [10, 10, 30, 40, 50]
```

PRACTICAL 7

Tuples

AIM

Write a Python program to perform all possible operations on tuples.

THEORY / CONCEPT

A tuple is an ordered, **immutable** collection of items defined using parentheses `()`. Despite being immutable, tuples support a wide range of operations: indexing, slicing, concatenation (+), repetition (*), membership testing (`in`), iteration, and built-in functions such as `len()`, `max()`, `min()`, `sum()`, and `sorted()`. Tuples also support unpacking, nested access, and methods like `count()` and `index()`. Attempting to modify a tuple element directly raises a `TypeError`, demonstrating their immutability.

PROGRAM / SOURCE CODE

```
# Practical 7: Tuples
# Perform all possible operations on tuples

# ---- Creating tuples ----
tuple1 = (10, 20, 30, 40, 50)
tuple2 = ('Python', 'Java', 'C++')
mixed_tuple = (1, "IoT", 3.5, True)
nested_tuple = (1, 2, (3, 4), [5, 6])
single_element_tuple = (7,)   # comma is required

print("1. Creating Tuples")
print("tuple1:", tuple1)
print("tuple2:", tuple2)
print("mixed_tuple:", mixed_tuple)
print("nested_tuple:", nested_tuple)
print("single_element_tuple:", single_element_tuple)
print()

# ---- Accessing elements ----
print("2. Accessing Elements (Indexing)")
print("tuple1[0]:", tuple1[0])
print("tuple1[-1]:", tuple1[-1])
print("tuple2[1]:", tuple2[1])
print()

# ---- Slicing ----
print("3. Slicing")
print("tuple1[1:4]:", tuple1[1:4])
print("tuple1[:3]:", tuple1[:3])
print("tuple1[::-1] (reverse):", tuple1[::-1])
print()

# ---- Concatenation ----
print("4. Concatenation")
concatenated = tuple1 + tuple2
print("tuple1 + tuple2:", concatenated)
print()

# ---- Repetition ----
print("5. Repetition")
repeated = tuple2 * 2
print("tuple2 * 2:", repeated)
print()

# ---- Membership test ----
print("6. Membership Test")
print("20 in tuple1:", 20 in tuple1)
print("'Java' in tuple2:", 'Java' in tuple2)
print("'Ruby' not in tuple2:", 'Ruby' not in tuple2)
print()

# ---- Iterating ----
print("7. Iterating through a Tuple")
for item in tuple2:
    print("Item:", item)
print()

# ---- Built-in functions ----
print("8. Built-in Functions")
print("len(tuple1):", len(tuple1))
print("max(tuple1):", max(tuple1))
print("min(tuple1):", min(tuple1))
print("sum(tuple1):", sum(tuple1))
print("sorted(tuple1, reverse=True):", sorted(tuple1, reverse=True))
print()

# ---- count() and index() ----
```

```
print("9. count() and index()")
tuple3 = (1, 2, 2, 3, 4, 2, 5)
print("tuple3:", tuple3)
print("tuple3.count(2):", tuple3.count(2))
print("tuple3.index(3):", tuple3.index(3))
print()

# ---- Tuple unpacking ----
print("10. Tuple Unpacking")
a, b, c, d, e = tuple1
print(f"a={a}, b={b}, c={c}, d={d}, e={e}")
first, *middle, last = tuple1
print("first:", first, " middle:", middle, " last:", last)
print()

# ---- Nested tuple access ----
print("11. Nested Tuple Access")
print("nested_tuple[2]:", nested_tuple[2])
print("nested_tuple[2][1]:", nested_tuple[2][1])
print()

# ---- Conversion ----
print("12. Conversion between list and tuple")
list_from_tuple = list(tuple1)
print("list(tuple1):", list_from_tuple)
tuple_from_list = tuple(list_from_tuple)
print("tuple(list_from_tuple):", tuple_from_list)
print()

# ---- Tuple immutability demonstration ----
print("13. Immutability Check")
try:
    tuple1[0] = 100
except TypeError as e:
    print("Error while modifying tuple:", e)
```

OUTPUT

```
1. Creating Tuples
tuple1: (10, 20, 30, 40, 50)
tuple2: ('Python', 'Java', 'C++')
mixed_tuple: (1, 'IoT', 3.5, True)
nested_tuple: (1, 2, (3, 4), [5, 6])
single_element_tuple: (7,)

2. Accessing Elements (Indexing)
tuple1[0]: 10
tuple1[-1]: 50
tuple2[1]: Java

3. Slicing
tuple1[1:4]: (20, 30, 40)
tuple1[:3]: (10, 20, 30)
tuple1[::-1] (reverse): (50, 40, 30, 20, 10)

4. Concatenation
tuple1 + tuple2: (10, 20, 30, 40, 50, 'Python', 'Java', 'C++')

5. Repetition
tuple2 * 2: ('Python', 'Java', 'C++', 'Python', 'Java', 'C++')

6. Membership Test
20 in tuple1: True
'Java' in tuple2: True
'Ruby' not in tuple2: True

7. Iterating through a Tuple
Item: Python
Item: Java
Item: C++

8. Built-in Functions
len(tuple1): 5
max(tuple1): 50
min(tuple1): 10
sum(tuple1): 150
sorted(tuple1, reverse=True): [50, 40, 30, 20, 10]

9. count() and index()
tuple3: (1, 2, 2, 3, 4, 2, 5)
tuple3.count(2): 3
```

```
tuple3.index(3): 3

10. Tuple Unpacking
a=10, b=20, c=30, d=40, e=50
first: 10  middle: [20, 30, 40]  last: 50

11. Nested Tuple Access
nested_tuple[2]: (3, 4)
nested_tuple[2][1]: 4

12. Conversion between list and tuple
list(tuple1): [10, 20, 30, 40, 50]
tuple(list_from_tuple): (10, 20, 30, 40, 50)

13. Immutability Check
Error while modifying tuple: 'tuple' object does not support item assignment
```

PRACTICAL 8

Dictionary

AIM

Write a Python program to create, add, delete, update and print items in a Dictionary.

THEORY / CONCEPT

A dictionary in Python is an unordered, mutable collection that stores data as **key-value** pairs enclosed in curly braces `{}`. New items are added simply by assigning a value to a new key, or using `update()` for multiple keys at once. Items can be removed using `del` or the `pop()` method (which also returns the removed value). Existing values can be updated by reassigning them to an existing key. Dictionaries provide the `keys()`, `values()`, and `items()` methods for iterating and printing their contents, and the `get()` method allows safe access with a default fallback value.

PROGRAM / SOURCE CODE

```
# Practical 8: Dictionary
# Create, Add, Delete, Update and Print items in a Dictionary

# ---- Creating a dictionary ----
student = {
    "name": "Aadhi",
    "course": "MCA",
    "university": "Parul University",
    "year": 2
}

print("1. Creating a Dictionary")
print("student:", student)
print()

# ---- Adding items ----
print("2. Adding Items")
student["email"] = "aadhi@paruluniversity.ac.in"
student["semester"] = 3
print("After adding 'email' and 'semester':")
print(student)
print()

# ---- Updating items ----
print("3. Updating Items")
student["year"] = 3          # single key update
student.update({"course": "M.Sc IT", "semester": 4})  # multiple key update
print("After updating 'year', 'course' and 'semester':")
print(student)
print()

# ---- Deleting items ----
print("4. Deleting Items")
del student["email"]
print("After del student['email']:")
print(student)

removed_value = student.pop("semester")
print(f"After student.pop('semester') -> removed value: {removed_value}")
print(student)
print()

# ---- Printing items in different ways ----
print("5. Printing Items")
print("Keys:", list(student.keys()))
print("Values:", list(student.values()))
print("Items:", list(student.items()))
print()

print("Iterating through dictionary (key : value):")
for key, value in student.items():
    print(f"{key} : {value}")
print()

# ---- Membership testing ----
print("6. Membership Testing")
print("'course' in student:", 'course' in student)
print("'email' in student:", 'email' in student)
print()

# ---- get() with default value ----
print("7. Using get() with a default value")
print("student.get('year'):", student.get('year'))
print("student.get('email', 'Not Available'):", student.get('email', 'Not Available'))
print()

# ---- Nested dictionary ----
```

```
print("8. Nested Dictionary")
university_data = {
    "MCA": {"duration": "2 years", "seats": 60},
    "BCA": {"duration": "3 years", "seats": 120}
}
print(university_data)
print("MCA seats:", university_data["MCA"]["seats"])
print()

# ---- clear() ----
print("9. Clearing a Dictionary")
temp_dict = {"a": 1, "b": 2}
print("Before clear:", temp_dict)
temp_dict.clear()
print("After clear:", temp_dict)
```

OUTPUT

```
1. Creating a Dictionary
student: {'name': 'Aadhi', 'course': 'MCA', 'university': 'Parul University', 'year': 2}

2. Adding Items
After adding 'email' and 'semester':
{'name': 'Aadhi', 'course': 'MCA', 'university': 'Parul University', 'year': 2, 'email': 'aadhi@paruluniversity.ac.in', 'semester': 3}

3. Updating Items
After updating 'year', 'course' and 'semester':
{'name': 'Aadhi', 'course': 'M.Sc IT', 'university': 'Parul University', 'year': 3, 'email': 'aadhi@paruluniversity.ac.in', 'semester': 4}

4. Deleting Items
After del student['email']:
{'name': 'Aadhi', 'course': 'M.Sc IT', 'university': 'Parul University', 'year': 3, 'semester': 4}
After student.pop('semester') -> removed value: 4
{'name': 'Aadhi', 'course': 'M.Sc IT', 'university': 'Parul University', 'year': 3}

5. Printing Items
Keys: ['name', 'course', 'university', 'year']
Values: ['Aadhi', 'M.Sc IT', 'Parul University', 3]
Items: [('name', 'Aadhi'), ('course', 'M.Sc IT'), ('university', 'Parul University'), ('year', 3)]

Iterating through dictionary (key : value):
name : Aadhi
course : M.Sc IT
university : Parul University
year : 3

6. Membership Testing
'course' in student: True
'email' in student: False

7. Using get() with a default value
student.get('year'): 3
student.get('email', 'Not Available'): Not Available

8. Nested Dictionary
{'MCA': {'duration': '2 years', 'seats': 60}, 'BCA': {'duration': '3 years', 'seats': 120}}
MCA seats: 60

9. Clearing a Dictionary
Before clear: {'a': 1, 'b': 2}
After clear: {}
```